

The NGRAM-pro family: why path-context coverage reaches deeper

Backdrop. *NGRAM-pro* collects the roadblocks where a fuzzer that keys coverage on *the last few edges it walked* (`naive_ngram4`'s 4-edge path-tuple feedback) gets through, while plain single-edge coverage (`naive`) stays stuck. The mechanism is not about *values* at all — unlike I2S (paste the right bytes) or VP (climb a per-byte gradient), n-gram coverage changes *what counts as new*. It folds the last four edge IDs into the map key, so the *same* basic block, reached through a different 4-edge path, lands in a *fresh* bucket and the seed that produced it is retained. Picture a maze where plain coverage only remembers “I’ve stood on this tile” — so it throws away the seed the second time it reaches a junction by a longer route — while n-gram remembers “I’ve stood on this tile *having come from there, via there, via there*”. That extra memory keeps the intermediate seeds that march a sequential decoder one step *deeper* each time; plain coverage collapses them to an already-seen edge and discards them, so its corpus never accumulates the depth the gate needs. Counts are validated roadblocks (*n*); a branch joins NGRAM-pro by its *deciding pair* (`naive_ngram4` resolves, plain `naive` blocks), not by the raw 4-char shape. The family is small and *honest* about it: one discriminable category, and two branches we leave *inconclusive* on principle (§2) because no built measurement can separate path-retention from plain luck there.

1. The path memory reaches a deeper decode state `ngram_sequential_depth_reach` (*n* = 2)

The whole validated family. The gate sits at the bottom of a *sequential* decode — a loop that consumes one code unit / one digit / one dispatch step at a time, with its own edge each pass. Each iteration through that region produces a new 4-edge path tuple, so `naive_ngram4` keeps the seed that got one step further in; plain `naive` sees the same block again, calls it “covered”, and drops the seed. Over the campaign the n-gram corpus drifts toward inputs that drive the decoder *deep enough to reach the gate*; the single-edge corpus stalls shallow. The tell is not “the right bytes are present” — it is “the n-gram seeds execute the blocker line far more / drive it much further in.”

```
// libxml2 encoding.c : UTF16LEToUTF8() -- assemble a UTF-16 surrogate
pair
while (in < inend) {
    c = *in++; c |= (*in++) << 8; // little-endian 16-bit code
    unit
    if ((c & 0xFC00) == 0xD800) { // <-- blocker (libxml2/6325):
        high surrogate
        d = *in++; d |= (*in++) << 8; // lines 534-545: only the
        winner gets here
        c = ((c & 0x3FF) << 10) + (d & 0x3FF) + 0x10000;
    }
}
```

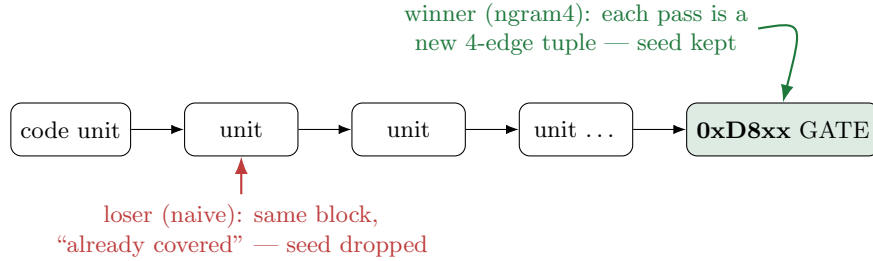
```

/* ... emit UTF-8 ... */                                // each pass is a fresh 4-edge
    path tuple
}

```

How we know it is really the path memory. We replay each side’s seed set through the instrumented binary and count, per seed, how many times the blocker line executes (its sequential-decode *depth*) and whether the seed takes the decisive side. The `naive_ngram4` arm should reach the line far more. Crucially this *discriminates*: a plain literal-failure story (“naive just lacks the right bytes”) predicts the loser reaching the *same* depth and failing the final compare (`depth_lift` $\approx$ 1); n-gram retention predicts the loser stalled *shallow*, never arriving.

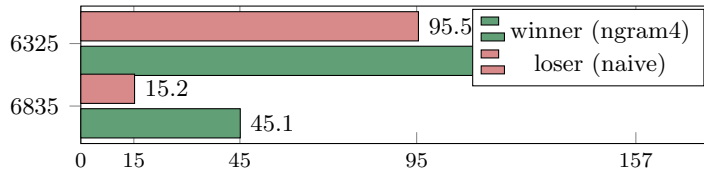
Tool: `depth_reach`
Metric: `depth_lift` = winner / loser per-seed executions of the blocker line; `side_lift` = how much more the winner takes the decisive branch; `winner_deeper` (bool)
Verify rule (the arbiter fires this, no LLM): `depth_lift` $\geq$ 1.5 AND `winner_depth` > `loser_depth`
Real example: libxml2/6325 (UTF16LEToUTF8, the high-surrogate gate (`c & 0xFC00`) == `0xD800`, `encoding.c:533`) — `winner_depth`= 156.7 vs `loser_depth`= 95.5, so `depth_lift`= 1.64 and `side_lift`= 1.55: the n-gram corpus drives the UTF-16 decode loop $\approx 1.6\times$ deeper per seed and into the surrogate-assembly lines 534–545 that the plain-naive corpus never reaches.



Plain coverage forgets the path, so it discards the seed that reached the junction by a longer route; the n-gram key remembers the route, keeps the seed, and the corpus walks one step deeper each generation until it reaches the gate.

Second member: a content-dispatch fan. The same memory, a different shape of “deeper.” Here the gate is the `'<![CDATA['` opener test (`cur[6] == 'T'`); the *winner* takes its *false* side and pours into the sibling content branches — end-tag `'</'`, PI `'<?'`, start-tag, char-data — each its own 4-edge tuple. The n-gram corpus keeps that *diversity* of non-CDATA seeds; plain coverage gives no extra credit for the path context, so those seeds are under-retained and short CDATA fragments dominate the loser’s corpus instead.

Tool: `depth_reach`
Metric: same tool, second branch — per-seed executions of the dispatch blocker line
Verify rule (the arbiter fires this, no LLM): `depth_lift` $\geq$ 1.5 AND `winner_depth` > `loser_depth`
Real example: libxml2/6835 (`xmlParseTryOrFinish`, the `'<![CDATA['` dispatch at `parser.c:11752`, operand `'T'`) — `winner_depth`= 45.1 vs `loser_depth`= 15.2, `depth_lift`= 2.97: the n-gram corpus reaches the content-dispatch chain $\approx 3\times$ more per seed. (`side_lift`= 0.9 here because the winner takes the dispatch *false* side — it is reaching the region more often, not crossing the same edge.)



per-seed blocker-line executions (decode depth) — winner = `naive_ngram4`, loser = `naive`

Both validated branches: the n-gram corpus drives the sequential blocker far deeper per seed (`depth_lift` = 1.64 and 2.97), clearing the ≥ 1.5 bar; the single-edge corpus stalls shallow.

2. Where we stop: two honest inconclusives

NGRAM-pro is deliberately the smallest *pro* family, and the reason is a property of the technique, not a gap in the data. n-gram coverage is a *non-local* feedback — its effect lives in *path tuples across the whole campaign*, not in any one branch’s bytes — so once a gate offers neither a *depth* gradient nor a *value* gradient, no branch-local measurement we have can separate “path-retention got us here” from “a luckier corpus got us here.” Per the project’s G3 guardrail (keep honest inconclusives; don’t chase 100%), we label these two and stop rather than force a story.

branch	gate	why no built tool discriminates
libxml2/6359	0xEFBBBF UTF-8 BOM	<i>Flat</i> 3-byte equality cascade (<code>in[0]==0xEF && in[1]==0xBB && in[2]==0xBF</code>): no decode loop, so <code>depth_reach</code> is <i>undefined</i> (no stage to climb). A Hamming-distance test <i>could</i> run, but a winner corpus closer to the BOM triple is equally consistent with plain literal-presence as with path-retention — not discriminating (fails G1).
libxml2/6887	<code>val < 0x800</code> overlong UTF-8	The loser <i>reaches the comparison at the same depth</i> (L T=0 F=1730: it runs the 3-byte decode block 1730×), so <code>depth_lift</code> ≈1 — depth cannot separate the arms. <code>value_distance_reached</code> to the 0x800 threshold was applied and <i>did not hold</i> (both arms sit at comparable magnitude distance); a relaxed re-test is forbidden.

In both cases a round-3 pivot over the remaining registry tools (`operand_enrichment`, `joint_necessity`, corpus-size / token-count) was rejected on principle: `operand_enrichment` needs a `cmplog` arm this `naive_ngram4-vs-naive` shape does not have, and the volume tools measure *whole-corpus* structure that a globally larger n-gram campaign would inflate anyway — so neither isolates path-tuple retention at the gate. The honest *inconclusive* is the correct outcome.

At a glance

Category	<i>n</i>	Tool / metric	One-line reason
sequential depth reach	2	<code>depth_reach</code> / <code>depth_lift</code> , <code>side_lift</code>	the 4-edge path memory keeps seeds that march the decoder deeper, so the n-gram corpus reaches the gate
NGRAM-pro (validated)	total 2		
honest inconclusive †	2	— (no discriminating tool)	flat BOM cascade (no depth gradient) / overlong inequality (equal depth, equal value-distance)

† Not part of the validated total. n-gram coverage is a *non-local* edge-tuple feedback: when a gate offers neither a depth nor a value gradient, no branch-local registry tool can separate path-tuple retention from a luckier corpus, so the branch is left **inconclusive** by design (G3).

Counts and example metrics regenerated from `bench/dataset.jsonl` and `step5b_new_v3/ngram_coverage_WL/{evidence_test,assignm`
the verify rule is the deterministic arbiter rule (no LLM in the decision). All four branches are libxml2; their corpora and source live on server sB and the gates are verified against the local source dumps there. The common thread is *path memory*: **naive_ngram4** treats a block reached by a longer route as new coverage, so it retains the intermediate seeds that walk a sequential decoder deep enough to reach the gate — depth the single-edge corpus collapses and forgets.